



Spoli Agency

Soluções em Automações

— GUIA COMPLETO · GIT & CLAUDE CODE

# Nunca mais quebre seu projeto com *Claude Code*

Controle de versão profissional do zero — commits, branches, conflitos e o fluxo de trabalho correto para vibecoders que usam o Claude Code.

 Git do zero

 Claude Code

 Comandos prontos

 Fluxo repetível

## CONTEÚDO

- 01 O que é Git e por que você precisa
- 02 Vocabulário essencial
- 03 Os 3 estados e o ciclo básico
- 04 Conventional Commits — o padrão do mercado
- 05 Branches — sua zona segura
- 06 Fluxo completo com Claude Code
- 07 Como resolver conflitos
- 08 Erros comuns e como escapar
- 09 Cheatsheet — referência rápida

# O que é Git — e por que você precisa *antes* de tudo

Git é o **histórico de versões do seu projeto**. Cada commit é um "salvar como" com data, hora e nota do que mudou. Se você ou o Claude Code quebrarem algo, é só voltar para a versão anterior.



## Repositório (repo)

A pasta `.git/` invisível dentro do seu projeto. Guarda todo o histórico — cada arquivo, cada linha, cada mudança.



## Commit

Um snapshot etiquetado do projeto num momento. Não tem como apagar — só acumula. É a base do versionamento.



## Branch

Uma linha do tempo paralela. Você testa features novas sem tocar no código que já funciona em `main`.



## Analogia prática

Git é como o histórico de versões de um documento do Google Docs. Você tem a versão estável em `main`, testa mudanças em cópias separadas (branches), e quando tudo funciona, mescla de volta. Se estragar algo, é só voltar para a versão anterior.



## Por que isso importa especialmente no Claude Code

O Claude Code faz commits automáticos enquanto trabalha. Por isso é essencial você entender o que ele está fazendo — para revisar as mensagens, corrigir o que ficou genérico e manter o histórico organizado. Sem Git, você perde o controle total do que foi mudado.

### — INICIAR GIT NUM PROJETO NOVO

```
# Entre na pasta do projeto e inicialize o Git
cd meu-projeto
git init

# O Git cria a pasta .git/ — seu repositório local
# Isso só é necessário uma vez por projeto
```

# Os termos que você precisa *conhecer*

Antes de usar qualquer comando, entenda o que cada termo significa. Isso elimina 80% da confusão inicial.

|                      |                                                                                                                                                                    |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>main / master</b> | Branch principal. O código de produção — o que vai para o ar.<br>Analogia: a versão publicada do app que o usuário vê.                                             |
| <b>HEAD</b>          | Ponteiro para onde você está agora no histórico. Indica o commit atual.<br>Analogia: o cursor numa planilha — indica a linha ativa.                                |
| <b>feature/xxx</b>   | Branch criada para desenvolver uma funcionalidade nova sem mexer na main.<br>Analogia: uma aba nova para testar fórmulas sem mexer na planilha original.           |
| <b>fix/xxx</b>       | Branch para corrigir um bug específico.<br>Analogia: uma cópia do documento para corrigir um erro sem afetar o original.                                           |
| <b>origin</b>        | Nome padrão do repositório remoto (GitHub). De onde você faz push e pull.<br>Analogia: o Google Drive onde você faz backup e compartilha o projeto.                |
| <b>staging area</b>  | Área de preparação — arquivos selecionados para entrar no próximo commit.<br>Analogia: o carrinho de compras. Você seleciona os itens antes de finalizar o pedido. |
| <b>merge</b>         | Operação de juntar o trabalho de uma branch com outra. Traz as mudanças para o destino.<br>Analogia: incorporar as edições do rascunho no documento principal.     |
| <b>push / pull</b>   | Push envia seus commits locais para o GitHub. Pull baixa os commits remotos para sua máquina.<br>Analogia: salvar no Drive (push) ou baixar do Drive (pull).       |

## Os 3 estados de um arquivo e o *fluxo do dia a dia*

Todo arquivo no Git vive em um de três estados. Entender isso elimina confusão na hora de fazer commits.



— FLUXO BÁSICO — RODE NESSA ORDEM

```
— SEMPRE COMECE AQUI —  
git status # veja o que mudou — rode SEMPRE primeiro  
  
— STAGING —  
git add . # adiciona tudo ao staging  
git add src/components/Header # ou um arquivo específico  
  
— COMMIT —  
git commit -m "feat: adiciona dashboard de gastos"  
  
— ENVIAR PARA O GITHUB —  
git push origin main  
  
— VER HISTÓRICO —  
git log --oneline # resumo compacto  
git log --oneline --graph --all # com grafo de branches
```



### Regra de ouro do commit

Um commit = uma mudança lógica. Se você precisa usar "e" na mensagem ("adiciona login e

corrige bug no dashboard"), faça dois commits separados. Isso facilita desfazer apenas uma mudança se precisar.

# O padrão de mensagens que o *mercado usa*

Conventional Commits é um padrão que torna o histórico legível por humanos e por ferramentas. Configure o Claude Code para segui-lo automaticamente.

## Formato da mensagem

**tipo**(*escopo opcional*): **descrição curta em minúsculas**

O **tipo** categoriza a mudança. O **escopo** é opcional e indica qual parte do sistema foi afetada. A **descrição** deve ser objetiva e em português.

### — EXEMPLOS REAIS DE USO

```
feat: adiciona tela de cadastro de despesas
feat(auth): implementa login com Google via Supabase
fix: corrige calculo incorreto do limite mensal
fix(dashboard): remove bug de renderizacao duplicada no grafico
chore: atualiza dependencias do package.json
docs: adiciona README com instrucoes de deploy
style: ajusta espacamento e cores da navbar
refactor: extrai logica de formatacao para utils/
perf: otimiza query de transacoes com index no Supabase
```

| TIPO     | QUANDO USAR                              | IMPACTO DE VERSÃO |
|----------|------------------------------------------|-------------------|
| feat     | Nova funcionalidade para o usuário       | minor version     |
| fix      | Correção de bug que afeta o usuário      | patch version     |
| chore    | Manutenção, configs, dependências        | sem versão        |
| docs     | Só mudanças em documentação              | sem versão        |
| style    | Formatação, espaços (sem lógica)         | sem versão        |
| refactor | Mudança de código que não é feat nem fix | sem versão        |
| perf     | Melhoria de performance                  | patch version     |



### Configure o Claude Code para seguir o padrão automaticamente

Adicione no seu `CLAUDE.md` : `"Use Conventional Commits em português. Sempre trabalhe em branches feature/ ou fix/ derivadas de main. Nunca commite direto na main."` Ele vai respeitar em toda sessão.

# Sua zona segura — nunca mais quebre a *main*

Mesmo sendo projeto solo, nunca commite direto na `main`. Se o Vercel está conectado à `main`, qualquer commit quebrado vai direto para produção.

## ❌ Sem branch — perigoso

Claude muda algo na `main` → deploy automático → usuário vê o erro → você corre para desfazer sem saber o que mudou.

## ✅ Com branch — seguro

Claude trabalha em `feature/xxx` → você testa localmente → só vai para a `main` quando funcionar → deploy tranquilo.

### — PADRÃO DE NOMENCLATURA DE BRANCHES

# PADRÃO: *tipo/descricao-com-hifens* (minúsculas, sem espaços)

#### — FEATURES

`feature/pagina-de-login`  
`feature/integracao-pagamento`  
`feature/tela-de-cadastro`

#### — CORREÇÕES

`fix/calculo-incorreto`  
`fix/redirect-apos-login`  
`hotfix/crash-na-tela-inicial` # bug crítico em produção

#### — MANUTENÇÃO

`chore/atualiza-dependencias`  
`docs/adiciona-readme-deploy`  
`refactor/reorganiza-estrutura-de-pastas`

#### — NUNCA COMMITTE DIRETO NESSAS

`main` # produção — conectado ao Vercel  
`develop` # staging (opcional, para equipes)

### — COMANDOS DE BRANCH

```
# Criar e já entrar na branch nova
git checkout -b feature/pagina-de-login

# Listar branches
git branch           # locais
git branch -a        # locais + remotas

# Trocar de branch
git switch main       # forma moderna (Git 2.23+)
git checkout main     # forma clássica

# Deletar branch após merge
git branch -d feature/pagina-de-login # local
git push origin --delete feature/pagina-de-login # remota
```

# O passo a passo completo — *do início ao merge*

Este é o fluxo que você deve repetir para cada feature ou correção. Siga sempre nessa ordem — sem pular etapas.

## 01 Atualize a main antes de começar

Nunca crie uma branch a partir de uma main desatualizada. Isso causa conflitos desnecessários depois.

## 02 Crie uma branch para a feature

Esta é a branch que o Claude Code vai usar durante o trabalho. Dê um nome descritivo.

## 03 Trabalhe com o Claude Code nessa branch

Abra o Claude Code e dê o contexto. Tudo ficará nessa branch — sem tocar na main.

## 04 Revise o que o Claude Code commitou

Confira as mensagens com `git log --oneline`. Se ficaram genéricas, corrija com `--amend`.

## 05 Teste localmente — depois merge para a main

Só faça o merge quando a feature estiver funcionando e testada. Nunca antes.

### — O FLUXO COMPLETO EM COMANDOS

```
— PASSO 1 — Atualizar main —————
git checkout main
git pull origin main

— PASSO 2 — Criar branch —————
git checkout -b feature/nova-funcionalidade

— PASSO 3 — Trabalhe com o Claude Code —————
# O Claude escreve código e faz commits automaticamente
# Você acompanha o que foi feito

— PASSO 4 — Revisar e corrigir commits do Claude —
git log --oneline
# f3a1b2c Add new feature ← mensagem ruim
git commit --amend -m "feat: adiciona nova funcionalidade"
# Para reescrever os últimos 2 commits interativamente:
git rebase -i HEAD~2

— PASSO 5 — Merge e limpeza —————
git checkout main
git merge feature/nova-funcionalidade
git push origin main
git branch -d feature/nova-funcionalidade # limpa a branch
```

# Como resolver conflitos

## sem *entrar em pânico*

Um conflito acontece quando duas branches modificaram a *mesma linha* de um arquivo de formas diferentes. O Git não sabe qual manter — ele pede pra você decidir. É normal, acontece com todo mundo.



### Como um conflito aparece no arquivo

O Git insere marcadores no arquivo. Tudo entre `<<<<<< HEAD` e `=====` é a sua versão. Tudo entre `=====` e `>>>>>>` é a versão da outra branch.

#### — COMO UM ARQUIVO CONFLITADO PARECE

```
// src/components/Header.tsx

<<<<<< HEAD           ← sua versão (branch atual)
const titulo = "Meu App";
=====               ← separador
const titulo = config.nomeDoApp;
>>>>>> feature/config-dinamica ← versão da outra branch
```

#### — PASSO A PASSO PARA RESOLVER

```
# 1. Ver todos os arquivos em conflito
git status
# both modified: src/components/Header.tsx

# 2. Abra o arquivo no VS Code
#   - o VS Code mostra botões "Accept Current" / "Accept Incoming"
#   - escolha qual versão manter e apague os marcadores <<<, ==, >>>

# 3. Marque como resolvido e finalize
git add src/components/Header.tsx
git commit -m "merge: resolve conflito no componente Header"

# Para ABORTAR e voltar ao estado anterior:
git merge --abort
```



## Os 4 erros mais frequentes — e como *escapar*

| # | ERRO                                        | SITUAÇÃO                                                      | SOLUÇÃO                                                                                                                     |
|---|---------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 1 | Commitou na branch errada                   | Fez commit na main mas deveria ser numa feature branch        | Crie a branch correta ( <code>git checkout -b feature/xxx</code> ), depois remova do main com <code>git reset HEAD~1</code> |
| 2 | Precisa desfazer o último commit            | O commit foi feito mas você se arrependeu                     | Use <code>git reset HEAD~1</code> (mantém arquivos) ou <code>git revert &lt;hash&gt;</code> (seguro para o remoto)          |
| 3 | git push rejeitado                          | "failed to push — remote contains work you do not have"       | Rode <code>git pull --rebase origin main</code> antes do push. Nunca use <code>--force</code> na main.                      |
| 4 | Mudanças não commitadas ao trocar de branch | Tem mudanças não commitadas e precisa mudar de branch urgente | Use <code>git stash</code> para guardar na "gaveta", troque de branch, depois <code>git stash pop</code>                    |

### — COMANDOS DE CORREÇÃO E STASH

#### — DESFAZER

```
git reset HEAD~1           # desfaz último commit, mantém arquivos
git reset --hard HEAD~1    # desfaz commit E apaga mudanças (cuidado!)
git revert <hash>          # cria commit que desfaz outro (seguro p/ remoto)
git commit --amend -m "nova msg" # corrige mensagem do último commit
```

#### — STASH (gaveta temporária)

```
git stash                 # guarda mudanças sem commitar
git stash push -m "wip: ajustes na tela" # com rótulo
git stash pop             # recupera e remove da gaveta
git stash list            # ver o que está na gaveta
```

#### — SINCRONIZAÇÃO

```
git pull --rebase origin main # integrar remoto sem merge commit extra
git fetch --all              # baixar metadados sem fazer merge
```

# Referência rápida —

## *todos os comandos*

Cole num lugar acessível enquanto está aprendendo. Esse é o vocabulário completo que você vai precisar.

### — REFERÊNCIA COMPLETA GIT

#### — CONFIGURAÇÃO (só uma vez) —

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

#### — STATUS E HISTÓRICO —

```
git status           # estado atual (sempre rode primeiro)
git log --oneline    # histórico resumido
git log --oneline --graph --all # grafo completo de branches
git diff             # mudanças ainda não em staging
git diff --staged    # mudanças em staging (pré-commit)
```

#### — STAGING E COMMITS —

```
git add .            # adiciona tudo ao staging
git add <arquivo>    # arquivo específico
git commit -m "feat: descricao" # commit com mensagem
git commit --amend -m "nova msg" # edita mensagem do último commit
git commit --amend --no-edit    # adiciona arquivo ao último commit
```

#### — BRANCHES —

```
git branch           # listar branches locais
git checkout -b feature/nome # criar e entrar na branch
git switch main      # trocar de branch (forma moderna)
git merge feature/nome # merge da branch para a atual
git branch -d feature/nome # deletar branch local (pós-merge)
git rebase main      # reaplica commits sobre o main
```

#### — REMOTO (GITHUB) —

```
git push origin main # envia main para o GitHub
git push -u origin feature/nome # primeira vez enviando branch nova
git pull origin main  # recebe atualizações do remoto
git pull --rebase origin main # integra sem criar merge commit
```

#### — DESFAZER E CORRIGIR —

```
git reset HEAD~1      # desfaz último commit (mantém arquivos)
git reset --hard HEAD~1 # desfaz commit E apaga mudanças
git revert <hash>      # cria commit que desfaz outro (seguro)
git restore <arquivo>  # descarta mudanças locais
git merge --abort      # cancela merge em andamento
```

#### — STASH (gaveta temporária) —

```
git stash           # guarda mudanças sem commitar
git stash pop       # recupera e remove da gaveta
git stash list      # ver o que está na gaveta
```



SPOLI AGENCY · SOLUÇÕES EM AUTOMAÇÕES

# Quer receber mais conteúdo assim sobre *IA todo dia?*

Seguindo o perfil da Spoli Agency você recebe guias práticos, tutoriais de Claude Code, automações e tudo que você precisa para produzir mais com IA.

 **Seguir @spoliagency**

Conteúdo gratuito. Sem spam. Só valor.

Claude Code

Git & GitHub

Automações com IA

Vibecoders

Deploy & Vercel